Machine Learning Quick Revision Notes + Code

---

1. Model Tuning

What is Model Tuning?

Finding the best hyperparameters to improve model performance.

Hyperparameters Examples:

```
RandomForestClassifier(
    n_estimators=100,
    max_depth=10
)
```

Why?

- Better accuracy
- Better generalization
- Avoid overfitting/underfitting

---

2. Cross Validation (CV)

Instead of one train-test split:

```
Data

Fold1 Test
Fold2 Test
Fold3 Test
```

```
Fold4 Test
Fold5 Test

Average Score
```

Code

```
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()

scores = cross_val_score(model, X, y, cv=5)

print(scores.mean())
```

---

3. Grid Search CV

Tests every combination.

Code

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

params = {
    'n_estimators':[50,100,200],
    'max_depth':[5,10,15]
}

grid = GridSearchCV(
    RandomForestClassifier(),
    params,
    cv=5
)
```

```
grid.fit(X_train,y_train)

print(grid.best_params_)
```

Formula

```
3 × 3 = 9 combinations
```

---

4. Randomized Search CV

Tests random combinations.

Code

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint
from sklearn.ensemble import RandomForestClassifier

params = {
    'n_estimators': randint(50,300),
    'max_depth': randint(5,20)
}

random = RandomizedSearchCV(
    RandomForestClassifier(),
    params,
    n_iter=20,
    cv=5
)

random.fit(X_train,y_train)

print(random.best_params_)
```

Interview

```
Grid Search = Exhaustive

Random Search = Faster
```

---

5. Ensemble Learning

Combining multiple models.

```
Weak Models
   
Combine
   
Strong Model
```

Types:

Bagging

```
Parallel Training
```

Example:

- Random Forest

Boosting

```
Sequential Training
```

Examples:

- AdaBoost
- Gradient Boosting

- XGBoost

Stacking

```
Multiple Models
      
Meta Model
      
Prediction
```

---

6. Random Forest

Idea

Many decision trees vote.

```
Tree1   Yes
Tree2   Yes
Tree3   No

Final = Yes
```

Code

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train,y_train)

pred = model.predict(X_test)
```

Advantages

- Less overfitting
- High accuracy
- Handles missing values

---

7. AdaBoost

Idea

Each model focuses on previous mistakes.

```
Model1
 
Errors
 
Model2
 
Errors
 
Model3
```

Code

```
from sklearn.ensemble import AdaBoostClassifier

model = AdaBoostClassifier(
    n_estimators=50,
    learning_rate=1
)

model.fit(X_train,y_train)
```

---

8. Gradient Boosting

Idea

Learns from residual errors.

```
Prediction Error

New Tree learns error

Repeat
```

Code

```
from sklearn.ensemble import GradientBoostingClassifier

model = GradientBoostingClassifier(
    n_estimators=100
)

model.fit(X_train,y_train)
```

---

9. XGBoost

Why Popular?

- Fast
- Regularization
- Handles missing values
- Wins many competitions

Code

```
from xgboost import XGBClassifier

model = XGBClassifier()

model.fit(X_train,y_train)
```

Interview

```
XGBoost = Optimized Gradient Boosting
```

---

10. Unsupervised Learning

Definition

No labels available.

```
Input Data
   
Find Hidden Patterns
```

Tasks:

- Clustering
- Dimensionality Reduction

---

11. K-Means Clustering

Steps

```
Choose K
   
Initialize Centroids
```

```
Assign Points

Update Centroids

Repeat
```

Code

```
from sklearn.cluster import KMeans

kmeans = KMeans(
    n_clusters=3,
    random_state=42
)

labels = kmeans.fit_predict(X)
```

---

12. Elbow Method

Find best K.

Code

```
wcss=[]

for k in range(1,11):
    model = KMeans(n_clusters=k)
    model.fit(X)

    wcss.append(model.inertia_)
```

Plot:

```
plt.plot(range(1,11),wcss)
```

Interview

```
Elbow Point = Best K
```

---

13. DBSCAN

Parameters

```
eps
min_samples
```

Point Types

```
Core Point
Border Point
Noise Point
```

Code

```
from sklearn.cluster import DBSCAN

db = DBSCAN(
    eps=0.5,
    min_samples=5
)

labels = db.fit_predict(X)
```

Advantages

- No K required
- Detects outliers
- Any cluster shape

---

14. PCA (Principal Component Analysis)

Goal

Reduce dimensions while preserving variance.

```
100 Features

PCA

10 Features
```

Steps

```
Standardize Data

Covariance Matrix

Eigenvectors

Principal Components
```

Code

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X_scaled = StandardScaler().fit_transform(X)

pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X_scaled)
```

---

15. Explained Variance

```
print(pca.explained_variance_ratio_)
```

Example:

```
PC1 = 72%
PC2 = 23%

Total = 95%
```

---

16. Curse of Dimensionality

Problem

As dimensions increase:

```
Distance becomes less meaningful
Data becomes sparse
Training becomes slow
Overfitting increases
```

Example:

```
2 Features    Easy

1000 Features    Hard
```

Solution

- PCA
- Feature Selection

Project 1: K-Means on Real Data

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

X,_ = load_iris(return_X_y=True)

kmeans = KMeans(n_clusters=3)

labels = kmeans.fit_predict(X)

print(labels)
```

Project 2: DBSCAN Complex Clusters

```
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN

X,_ = make_moons(
    n_samples=500,
    noise=0.05
)

db = DBSCAN(
    eps=0.2,
    min_samples=5
)
```

```
labels = db.fit_predict(X)
```

Why DBSCAN?

```
K-Means fails on moon shape

DBSCAN succeeds
```

Project 3: PCA Visualization

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X,y = load_iris(return_X_y=True)

X = StandardScaler().fit_transform(X)

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X)
```

Plot:

```
plt.scatter(
    X_pca[:,0],
    X_pca[:,1],
    c=y
)
plt.show()
```

---

10 Interview One-Liners

1. **Cross Validation** Evaluates model on multiple folds.
2. **GridSearchCV** Tries all parameter combinations.
3. **RandomizedSearchCV** Tries random combinations.
4. **Bagging** Parallel learners.
5. **Boosting** Sequential learners correcting errors.
6. **Random Forest** Collection of decision trees.
7. **AdaBoost** Focuses on misclassified samples.
8. **K-Means** Partitions data into K clusters.
9. **DBSCAN** Density-based clustering with outlier detection.
10. **PCA** Reduces dimensions while preserving maximum variance.

Last-Minute Revision Flow

Model Tuning

Cross Validation

GridSearchCV / RandomizedSearchCV

Ensemble Learning

Random Forest

AdaBoost

Gradient Boosting

XGBoost

Unsupervised Learning

K-Means + Elbow

DBSCAN

PCA

Curse of Dimensionality